

MyOffice Classic Design System Archive

Principles, architecture, component contracts, lessons, and retirement guidance

Status: archival reference for the Dallaglio-only migration

Source snapshot: frontend commit dd75b27 on 29 September 2026

Audience: future MyOffice maintainers, designers, auditors, and coding agents

Runtime authority after retirement: Dallaglio only

Executive decision

The Classic MyOffice design system is being retired as a runtime option. Its visual language—frosted glass, violet gradients, colored glow, filled icon treatment, legacy width constraints, and Classic-specific component branches—must not remain selectable or leak into the Dallaglio interface.

The work invested in Classic is not discarded. This document preserves the system's useful design and engineering principles, behavioral contracts, accessibility safeguards, component responsibilities, implementation history, and known failure modes. Those principles should be translated into the Dallaglio system where they remain valid. The old visual flavor and compatibility machinery should be removed after the translation is verified.

The distinction is deliberate:

- **Retain:** product behavior, accessibility, information hierarchy, semantic state meaning, reusable component contracts, preference behavior that still serves users, interaction clarity, resilient data-state principles, and verification discipline.
- **Translate:** useful interaction patterns whose Classic implementation is visually obsolete, such as card affordances, modal structure, grouped records, icon meaning, motion sequencing, and contextual actions.
- **Retire:** Classic tokens, Classic icon maps, Classic-only markup, design selection, [myoffice_design](#) persistence, Studio and Paper compatibility bases, Classic smoke tests, and any route branch whose only purpose is to preserve the old visual language.

This archive is documentation only. It must never be imported by the application, used as a source of runtime styles, or treated as permission to reintroduce Classic.

1. What Classic was

Classic was the original shared MyOffice visual system, also described historically as the Studio glass or glass and glow language. It grew from a collection of page-level patterns into a central design system under [components/shared/design-system/](#).

Classic combined five concerns:

- 1. Appearance selection.** A persisted `classic | dallaglio` preference changed the shell, shared components, and route-level markup.
- 2. Theme tokens.** Light and dark values supplied surface, text, border, hover, input, link, shadow, and action classes.
- 3. Reusable components.** Shared cards, heroes, stat tiles, forms, dialogs, record rows, search controls, action buttons, and empty states reduced page-by-page duplication.
- 4. Interaction and motion.** Shared Framer Motion variants, reduced-motion handling, hover intent, count-up animation, modal behavior, and scroll affordances standardized feedback.
- 5. Compatibility.** Route branches and CSS bases allowed the old Studio appearance to coexist with the newer Dallaglio language while preserving business behavior.

Classic was not merely a stylesheet. It was an architectural layer that connected providers, local storage, the document root, the app shell, shared components, route markup, icon registries, charts, tests, and browser verification scripts.

2. Design lineage and terminology

The repository contains three related historical names:

Name	Meaning at the archived snapshot	Retirement treatment
Studio	Early name for the original MyOffice glass and glow language. <code>bases/studio.css</code> only identifies the base through <code>--ds-name: studio</code> .	Retire the CSS base and aliases. Preserve its principles here.
Paper	Early Tools-derived global restyle activated by <code>data-design='paper'</code> . It introduced flatter surfaces, semantic variables, and overrides for leftover Studio gradients.	Retire as superseded migration scaffolding. Its useful restraint already belongs in Dallaglio.
Classic	Runtime name for the original Studio appearance after the formal Classic and Dallaglio split.	Remove from runtime after principle translation and verification.
Dallaglio	Tools-derived design language with semantic tokens, restrained structure, scan-first layouts, and its own controls, icons, and shell.	Sole runtime design after retirement.

Light and dark appearance are not design languages. They remain valid user preferences after Classic retirement.

3. Non-negotiable principles to carry forward

3.1 One owned system, not page-level styling islands

The strongest Classic contribution was architectural: pages should consume shared tokens and components instead of copying utility strings and rebuilding controls locally.

Future rules:

- Search the shared design-system barrel before creating a component.
- Extend an existing component when behavior is the same and only a meaningful option differs.
- Keep business logic in route or domain code; keep visual and interaction behavior in shared primitives.
- Fix recurring defects at the shared layer so every route benefits.
- Do not introduce another competing component framework without a documented architecture decision.

3.2 Meaning before decoration

Classic used color heavily, but the mature system separated color responsibilities:

- **Structure:** canvas, surfaces, text, borders, navigation, controls, and active states.
- **Semantic state:** success, warning, danger, information, compliance, approval, and operational status.
- **Data:** chart series, comparisons, heat maps, and visualization highlights.
- **Brand:** approved identity and primary interaction emphasis.

Dallaglio should retain this separation. In dark mode, structural color remains black, white, and graphite. Semantic and chart color remains available because it conveys meaning. Decorative rainbow treatment must not return through translated Classic components.

3.3 Behavior survives visual replacement

When a Classic branch is removed, its business behavior must remain unchanged unless a separate product decision says otherwise. This includes handlers, permission checks, form validation, filters, sorting, exports, persistence, dialog outcomes, keyboard interactions, and semantic status labels.

A visually cleaner Dallaglio replacement is not complete if it drops an old affordance or changes a workflow.

3.4 Scan-first information hierarchy

Dense operational pages must allow users to identify the record, state, owner, time, and next action quickly. The mature component set encoded this through:

- page heroes that separate title, context, actions, and key statistics;
- grouped sections for naturally related records;
- record cards with stable summaries and expandable detail;
- status badges only where state needs to be read at rest;
- contextual actions that do not compete with primary data;
- full-width layouts for dense operational routes where the viewport can improve comprehension.

The Dallaglio translation should preserve the hierarchy while using its own spacing, surfaces, typography, and icons.

3.5 Accessible primitives over custom imitation

Classic's later architecture replaced ad hoc dialog behavior with Radix primitives while preserving appearance. The enduring rule is to keep native semantics and accessible library behavior:

- focus trapping and restoration;
- `role='dialog'` and `aria-modal`;
- Escape behavior that respects nested selects and popovers;
- meaningful button labels and `aria-label` text;
- `aria-pressed` for selectable controls;
- minimum usable touch targets;
- keyboard-visible focus;
- reduced-motion compliance;

- native date and time pickers opened from the whole input field where supported.

Do not replace these behaviors with hand-built event handlers merely to obtain a visual effect.

3.6 Truthful states

Loading, unavailable, empty, error, and retry are different states. A failed request must not appear as an empty register. Stale records must be cleared when a request definitively fails, and out-of-order responses must not overwrite newer user choices.

This principle came from the wider Dallaglio audit but applies to every component translated from Classic. Visual retirement must not weaken data truthfulness.

3.7 Verify rendered behavior, not only source

Source review, tests, type checks, lint, builds, and browser verification are separate evidence categories. A migration is not visually complete until the relevant routes are rendered in both light and dark modes at desktop and phone widths, with key interactions exercised.

4. Runtime architecture at the archived snapshot

4.1 Provider stack

`components/Providers.tsx` mounted the following providers inside authentication:

1. `ThemeProvider`
2. `IconStyleProvider`
3. `FontStyleProvider`
4. `FontScaleProvider`
5. `ConfirmProvider`

This composition made appearance, icon weight, body font, interface scale, and confirmation behavior available throughout the application. The nesting order was explicit and centralized.

Translation rule: preserve the provider responsibilities that remain useful, but remove the design-language selector. Light and dark theme, font preference, font scaling, icon consistency, and confirmation behavior may continue if they conform to Dallaglio.

4.2 Design persistence and document attributes

The archived design contract lived in `components/shared/design-system/shared/design.ts`:

```
export type DesignLanguage = 'classic' | 'dallaglio';
export const DESIGN_KEY = 'myoffice_design';
export function readDesignLanguage(): DesignLanguage;
export function persistDesignLanguage(design: DesignLanguage): void;
export function isToolsPath(pathname: string | null | undefined): boolean;
export function applyDesignToDocument(design: DesignLanguage): void;
```

Historical `studio` values were interpreted as Classic. The document root received `data-design`, and the Tools route was forced to Classic during the transition.

`app/layout.tsx` duplicated the critical selection logic in a pre-paint script so React hydration would not cause an appearance flash. It also applied theme and font scale before first paint.

Retirement rule: remove `DesignLanguage`, `DESIGN_KEY`, `readDesignLanguage`, `persistDesignLanguage`, Tools force-Classic logic, and design-specific pre-paint code. Remove stale `myoffice_design` values from browser storage during migration. Continue applying light or dark theme and font scale before paint.

4.3 Theme contract

themeClasses(light, design) returned a stable semantic shape for both systems:

Token	Responsibility
glass	primary panel or card surface
glassSoft	lighter secondary surface
glassPopover	high-contrast overlay surface
shadow	ambient elevation
textPrimary	strongest content text
textSecondary	supporting text
textTertiary	lower-priority text
textFaint	labels and tertiary metadata
textMuted	readable secondary text
border	standard boundary
divide	list and section dividers
hoverBg	stronger hover background
hoverBgSoft	subtle hover background
hoverText	interactive text emphasis
groupHoverText	parent-hover text emphasis
chipBg	compact control background
inputBg	complete input appearance
trendUp	positive trend text
trendDown	negative trend text
ring	ring contrast
scrim	overlay background
linkText	link color
linkHover	link hover color
pageBg	route canvas
cta	primary action
ctaDanger	destructive action

Classic light surfaces were opaque white with stone or gray boundaries. Classic dark surfaces used near-black translucent glass, backdrop blur, pale text, subtle white borders, and layered ambient shadows. Popovers were more opaque than large glass panels to prevent text beneath them from bleeding through.

Translation rule: keep the semantic contract, not the glass implementation. Dallaglio tokens should remain the only implementation, and their naming may be modernized when all call sites are migrated together.

4.4 User preferences

The archived system persisted:

Key	Preference
myoffice_theme	system, light, or dark
myoffice_design	Classic or Dallaglio selection
oz_bodyFont	System, Inter, Manrope, Jakarta Sans, or Sora
oz_fontScale	Small, default, large, or extra large
oz_iconStyle	Solid or outline icon weight

The preferences panel also stored default list view, initial section expansion, and typed input history through separate preference helpers.

Translation rule: remove only the design selector and obsolete icon treatment if Dallaglio standardizes one icon weight. Preserve user-centered controls that remain coherent and accessible. Settings must have one obvious home rather than duplicate controls scattered through the top bar and bottom bar.

5. Classic visual language

This section records the old appearance for historical completeness. It is not a Dallaglio implementation specification.

5.1 Surfaces

Classic dark mode used near-black translucent surfaces such as #161614 with 65 to 80 percent opacity, white borders near 8 percent opacity, and backdrop-blur-lg or backdrop-blur-xl. Popovers used a near-opaque #121210 surface to protect readability.

Classic light mode used opaque white or nearly opaque white surfaces, stone or gray borders, and restrained neutral shadows.

5.2 Elevation

The system combined:

- ambient resting shadows;
- subtle inset highlights and lower edges to suggest material depth;
- hover lift of approximately five to six pixels;
- delayed colored glow based on a card's accent or semantic color;
- optional stronger resting elevation for deliberately prominent tiles.

5.3 Brand treatment

Classic primary actions used violet gradients and white text. The shell mark used a three-stop violet gradient with inset highlight, lower shading, and a colored drop shadow. Brand violet also appeared in links, chips, selected states, and some notification cues.

5.4 Typography

Headings used Montserrat. Body text was user-selectable, with Manrope as the archived default and Inter, Jakarta Sans, Sora, or system UI available. A separate interface scale changed the whole page through CSS zoom because many routes used pixel-based text sizes.

5.5 Iconography

Classic used a central Phosphor alias barrel and defaulted to solid icons through `IconContext`. Users could switch globally to outline icons. Logical names shielded routes from package-specific glyph names.

Classic's design-specific icon registry mapped semantic meanings—such as employees, maintenance, warning, compliance, and overdue—to aliases from the shared icon barrel. Dallaglio used a separate thin light-weight registry.

Translation rule: keep meaning-based icons and one central mapping. Retire the Classic map and user-facing solid/outline choice if it conflicts with Dallaglio consistency. Do not let routes import icon packages directly.

6. Shared interaction and component contracts

6.1 Button

The shared button supported:

```
type ButtonVariant = 'primary' | 'secondary' | 'subtle' | 'ghost' | 'danger' | 'icon';
type ButtonSize = 'xs' | 'sm' | 'md';
```

It normalized loading, disabled state, icon placement, links versus buttons, full-width behavior, pressed state, and accessible labeling. Classic and Dallaglio supplied different visual classes behind the same API.

Retain: one semantic button API, explicit destructive treatment, icon placement, loading state, and accessible pressed state. **Retire:** `classicClasses` and visual branching.

6.2 CloseButton

The shared close control standardized icon size, click target, focus behavior, and design-specific styling. It replaced small page-local X buttons, especially in dialogs.

Retain: a consistent, sufficiently large dismiss target with an accessible name. **Retire:** Classic styling and branch logic.

6.3 GlowCard

Classic's signature card primitive combined a shared surface, resting depth, hover lift, delayed glow, optional persistent glow, optional elevated rest state, and reduced-motion behavior.

The component also encoded a key design lesson: do not stack thick colored borders or ribbons on a card merely to show category or status. Those decorations compete with content and the hover affordance. Persistent state belongs in a badge or icon; selection rings are acceptable only when they are the functional selection indicator.

Retain: one card primitive, clear click affordance, restrained motion, selected-state clarity, and the rule against redundant decorative status signals. **Translate:** Dallaglio should express affordance through line, contrast, shadow, and contextual interaction rather than Classic colored glow. **Retire:** glass, multi-layer inset bevels, and colored hover glow.

6.4 EmptyState and LoadingState

The shared empty state provided an icon, title, optional explanation, and optional action. The loading state standardized spinner and label.

Retain: descriptive, actionable, and truthful states. Empty must mean a successful query with no records; unavailable must include retry where possible.

6.5 CenterModal

The modal facade used Radix Dialog for focus trapping, portal rendering, Escape behavior, and semantic attributes. Classic added blurred scrim, glass panel, accent glow, animated scale and vertical motion, shared close control, title, subtitle, and scrollable content.

Nested controls used `handleModalEscapeKeyDown` so Escape closed a nested select or popover before closing the entire modal.

Retain: the accessible behavioral contract, z-index discipline, scroll boundaries, stable title and subtitle hierarchy, and nested Escape handling. **Retire:** Classic glass and accent glow.

6.6 PageHero

The page hero standardized breadcrumbs, icon, title, description, action placement, and a collapsible statistics strip. It kept page identity and primary action in a predictable location.

Retain: consistent structure, responsive action wrapping, and optional statistics. **Translate:** use Dallaglio typography, surface, and icon weight only.

6.7 StatTile, StatCard, StatStrip, and ProgressBar

These components standardized summary metrics, animated values, labels, optional trend or status cues, and progress display.

Retain: consistent metric hierarchy, correct units, explicit status meaning, and reduced-motion handling. **Translate:** decorative color becomes neutral in Dallaglio dark mode; semantic color remains. Charts use their own palette.

6.8 GroupSection and Subsection

Grouped lists reduced long flat scans. `GroupSection` supplied a collapsible category container; `Subsection` supplied a lighter nested grouping without adding another heavy surface.

The implementation guidance was to use nested subgroups only when they meaningfully consolidate records. A set of one-record subgroups adds noise rather than structure.

Retain: meaningful grouping, disclosed counts, keyboard-operable expansion, and preference-aware initial state.

6.9 RecordCard, SummaryItem, InfoRow, and actions

The record pattern separated:

- always-visible summary fields;
- expanded details;
- semantic status badges;
- edit, view, delete, and disclosure controls;
- record-level click behavior from nested action behavior.

`InfoRow` represented labeled detail values. `SummaryItem` represented compact icon, label, and value lines.

`RecordActions`, `DetailActions`, and `DisclosureButton` standardized actions and accessible text.

Retain: stable summaries, progressive disclosure, clear action scope, event propagation discipline, and touch availability. **Retire:** Classic icons, glass, glow, and branch-specific action markup.

6.10 Forms and selection controls

`FormField`, `FormActions`, `AutofillInput`, `SearchInput`, `SelectField`, `Combobox`, and `ViewToggle` carried reusable behavior beyond appearance.

Important retained principles:

- labels remain visible and associated with controls;
- required state is explicit;
- autofill suggestions are keyboard and pointer accessible;
- suggestion selection must reliably populate the field;
- portaled lists preserve wheel scrolling and edge feedback;
- date and time inputs open their native picker from the full field when supported;
- selected view or filter states expose `aria-pressed` or equivalent semantics;
- touch targets remain usable on coarse pointers;
- form actions have consistent cancel and submit ordering.

6.11 Confirmation

`ConfirmProvider` and `useConfirm` replaced browser confirmation with a consistent asynchronous contract.

Retain: explicit title, explanation, cancel, confirm, destructive styling when appropriate, keyboard support, and a promise-based calling API. **Retire:** Classic surface styling.

7. Motion principles

Classic defined shared easing and variants rather than route-specific animation objects:

- `EASE_OUT = [0.22, 1, 0.36, 1]`
- `EASE_SOFT = [0.16, 1, 0.3, 1]`
- shared `fadeUp`, `staggerContainer`, `fadeTextVariant`, tile icon and text variants, and icon pop behavior;
- dropdown entrance through a shared class;
- count-up animation for metrics;
- motion disabled or shortened for `prefers-reduced-motion`.

The sequencing principle was intentional: the main interaction response should happen first, decoration second, and supporting text last. In Classic cards, lift occurred quickly, glow built more slowly, and text emerged afterward.

Translation rule: preserve calm sequencing and reduced-motion support, but do not preserve motion solely to imitate Classic. Dallaglio may use less motion. Every animation must communicate hierarchy, state change, spatial relationship, or direct manipulation.

8. Icon principles

The mature icon system solved several consistency problems:

1. Routes imported logical names from the shared barrel, not package glyphs directly.
2. A meaning registry allowed design-specific glyph maps without changing business components.
3. Common actions had one standard glyph: edit, delete, add, call, search, filter, and similar actions did not vary by page.
4. Category icons stayed distinct within the same module group.
5. Semantic status colors were allowed; decorative category color was constrained.
6. Icons inherited current color and scale from the surrounding control.

Dallaglio-only target: keep the logical meaning registry and Dallaglio map. Remove `CLASSIC_ICONS`, Classic size and weight decisions, dual-map coverage tests, and the global solid or outline preference if Dallaglio mandates light-weight icons.

9. Color principles and archived palettes

9.1 Accent palette

The shared system exposed six named accents:

- blue;
- amber;
- indigo;
- emerald;
- cyan;
- violet.

It also exposed hex and rgba forms, text variants, gradients, and a default brand glow. These values served Classic decoration, category distinction, and some shared APIs.

9.2 Semantic palette

`STATUS_TONE` separated operational meaning from arbitrary accent choice. A helper detected semantic hex values so components could preserve meaningful color in dark mode while suppressing decorative color.

9.3 Charts

`chartTheme(design, light)` supplied visualization colors independently from UI interaction color. This prevented an achromatic Dallaglio interface from producing monochrome charts and prevented Classic brand violet from becoming the only data series.

Retain: one shared chart theme, categorical distinction, semantic overlays, legible grid and tooltip contrast, and no derivation from button or primary-action colors. **Retire:** the `design` parameter and Classic chart branch after all call sites use the single Dallaglio chart contract.

10. App shell responsibilities

The Classic and Dallaglio split reached the full shell:

- `AppShell` selected canvas and shell module styles.
- `TopNavigation` changed search chrome, logo mark, tools link, design switch, icon-style control, settings controls, and shadows.
- `SidebarNavigation` changed surfaces, active treatment, icon treatment, favorites, and collapse affordances.
- `BottomBar` changed blur, shadow, menu surfaces, and settings presentation.
- `AuthMenu` changed avatar rings and shared surface tokens.
- `PreferencesPanel` exposed the design selector and rendered its own segmented-control branches.

The shell also provided valuable behavior unrelated to Classic:

- module search and recent-search history;
- notifications and unread counts;
- responsive navigation;
- favorites and dashboard customization;
- sidebar collapse persistence;
- signed-in account, role, security, and sign-out access;
- a delegated native date and time picker enhancement;
- first-run preferences;

- quick actions and active notices.

Retirement rule: preserve the behavior, replace every visual branch with the Dallaglio implementation, remove design-switch controls, and ensure no Classic-specific CSS class or token remains in shell output.

11. Responsive and layout principles

Classic commonly constrained page content to route-specific maximum widths such as 1100, 1400, or Tailwind's `max-w-7xl`. Dallaglio later adopted full-width layouts on dense routes, with the Classic constraints preserved through conditional branches.

The durable rule is not “always full width.” It is:

- use the available viewport when additional width improves scanning or reduces destructive wrapping;
- apply deliberate readable measures to prose and narrow forms;
- keep stable page padding at phone, tablet, and desktop widths;
- prevent fixed columns, sticky headers, and intrinsic native controls from overflowing;
- verify layouts at approximately 390 pixels and at representative desktop widths.

When Classic branches are removed, retain Dallaglio's full-width operational layout and keep narrower measures only where content type justifies them.

12. Accessibility and native-control safeguards

12.1 Reduced motion

`usePrefersReducedMotion` observed the operating system preference and disabled decorative loops or transformed animation. CSS modules also included reduced-motion media queries.

12.2 Focus and keyboard behavior

Shared controls used visible focus treatment, semantic buttons, pressed state, dialog focus trapping, keyboard dismiss, and accessible labels. Dallaglio must keep those behaviors after Classic markup is removed.

12.3 Date and time inputs

The old system recorded two recurring defects:

1. A globally forced dark `color-scheme` can make the browser's native picker icon unreadable in light mode. Scope date and time input color scheme to the current theme.
2. Two native date inputs inside a narrow fixed grid cell can overflow because their intrinsic width cannot shrink freely. Give the range an appropriate grid span and apply `min-width: 0` where needed.

12.4 Touch and pointer behavior

Actions hidden until hover must remain visible or otherwise discoverable on coarse pointers. Destructive and dismiss controls need adequate targets and unambiguous accessible names.

12.5 Popovers and scroll

Portaled lists need robust wheel scrolling, overscroll containment, and edge feedback. Overlay surfaces must be opaque enough that underlying text cannot reduce readability.

13. Known mistakes and lessons

The following defects were encountered during the system's evolution and should remain part of the project's institutional knowledge:

- Migrating the first card in a file did not mean every repeated card was migrated. Category tiles, folders, filters, list rows, and alternate views were repeatedly missed.
- Thick colored edge ribbons on cards duplicated status communication and overwhelmed the intended hover affordance.
- Flat gray icons next to strongly colored content created inconsistent hierarchy.
- A medium primary button once lost horizontal padding, causing text to approach or overlap its edges.
- Small page-local icon buttons created inconsistent hit targets and duplicated action semantics.
- A dropdown result could disappear on input blur before its click fired; handling the committed action on pointer down prevented the race.
- A dialog trigger nested directly inside a closing menu could race the menu close; controlling the dialog separately avoided the conflict.
- Low-opacity glass was unsuitable for small popovers placed above dense text.
- Unscoped native input `color-scheme` broke picker icons across themes.
- CSS branch overlays could cosmetically mask old components without actually retiring their markup or behavior.
- Source success was repeatedly mistaken for visual completion. Rendered verification must remain an explicit gate.

14. Principle translation matrix

Classic concept	Underlying principle	Dallaglio translation	What must disappear
Glass panels	Group content and separate hierarchy	Solid Dallaglio semantic surfaces and lines	blur, translucent glass, Classic surface constants
Colored glow card	Signal interactivity and elevation	restrained border, contrast, shadow, contextual affordance	colored glow, inset bevel stack, Classic motion branch
Violet gradient CTA	Make the primary action obvious	one Dallaglio primary button style	gradient classes and Classic CTA branch
Filled or outline icon preference	Maintain icon consistency	one Dallaglio icon weight and meaning registry	<code>oz_iconStyle</code> , global style toggle if no longer needed
Classic and Dallaglio icon maps	Decouple meaning from glyph	retain Dallaglio map only	<code>CLASSIC_ICONS</code> , dual-map tests
Classic popover opacity rule	Preserve readability over dense content	opaque Dallaglio popover surface	Classic glass popover token
GlowCard sequence	Give direct feedback before decoration	faster structural response, minimal supporting motion	Classic glow timing
Page maximum widths	Protect readability	full-width operational pages plus local readable measures	design-dependent width ternaries
Colored category tiles	Make groups distinguishable	neutral structure, selective categorical data color	decorative dark-mode rainbow surfaces
Status pills	Communicate state at rest	Dallaglio semantic badges	decorative or duplicate badge color
PageHero	Stable page identity and actions	Dallaglio hero only	Classic hero branch
CenterModal	Accessible focused task flow	Dallaglio modal facade over Radix	Classic scrim, glow, radius branch
GroupSection	Reduce scanning load	Dallaglio grouped disclosure	Classic panel visuals

Preferences design control	Let users choose visual flavor	no design choice; retain useful accessibility preferences	selector, storage key, switch buttons
Classic smoke scripts	Protect backward compatibility	Dallaglio light and dark regression checks	Classic browser phases and screenshots

15. Retirement implementation checklist

This checklist defines completion for the later code migration.

15.1 Central contract

- Remove `DesignLanguage` and make Dallaglio the only theme implementation.
- Remove `DESIGN_KEY`, read and persist helpers, and `myoffice_design` pre-paint logic.
- Remove Tools force-Classic behavior.
- Remove `design` and `setDesign` from `Theme` and all consumers.
- Keep light, dark, system theme, and useful accessibility preferences.

15.2 Shared visual infrastructure

- Remove Classic `GLASS`, `GLASS_SOFT`, `GLASS_POPOVER`, light glass, and Classic shadow constants.
- Make the semantic theme token contract resolve only to Dallaglio.
- Remove `classic/`, `bases/studio.css`, and `bases/paper.css` after confirming they have no imports.
- Remove Classic selectors and obsolete `oz-glass-*` utilities when their call sites reach zero.
- Remove Classic chart palettes and the chart design parameter.

15.3 Icons and preferences

- Make `DslIcon` use only the Dallaglio registry.
- Remove `CLASSIC_ICONS` and Classic coverage assertions.
- Remove Classic and Dallaglio switch buttons from the top navigation and preferences panel.
- Decide whether the solid or outline icon preference remains useful; if not, remove `IconStyleProvider`, `oz_iconStyle`, and its controls.
- Keep logical icon meanings and centralized mappings.

15.4 App shell

- Collapse `AppShell`, `TopNavigation`, `SidebarNavigation`, `BottomBar`, `AuthMenu`, and `PreferencesPanel` to their Dallaglio branches.
- Remove Classic backgrounds, shadows, marks, rings, and segmented controls.
- Preserve search, notifications, customization, preferences, account, security, responsive behavior, and native picker enhancement.

15.5 Shared components

- Collapse Button, CloseButton, cards, modal, empty state, status, search, selection, record, detail, and action components to Dallaglio implementations.
- Preserve all exported contracts until call sites are migrated; simplify APIs only with type-safe call-site updates.
- Keep Radix and native semantics.

15.6 Route branches

- Replace every `t.design === 'dallaglio'` and inverse branch with the Dallaglio result.
- Remove design-dependent width choices.
- Preserve handlers, domain constants, validation, status names, calculations, exports, and mutation paths.
- Re-check alternate views and nested dialogs, not only primary cards.

15.7 Tests and scripts

- Replace Classic token and persistence tests with Dallaglio-only invariants.
- Remove Classic smoke phases from browser scripts.
- Test light and dark independently from design selection.
- Add a static check that forbids runtime references to `classic`, `studio`, `paper`, `myoffice_design`, and `setDesign`, with explicit exclusions for this archive and historical audit records.

15.8 Documentation

- Mark the dual-system handoff as superseded.
- Update active architecture and design-system references to a single Dallaglio runtime.
- Preserve historical audit entries without rewriting what was true when they were recorded.
- Add a dated audit section documenting the retirement and actual verification evidence.

16. Verification contract for retirement

16.1 Static evidence

The runtime source should contain no design selector or Classic implementation. Searches should return zero applicable matches for:

```
myoffice_design
setDesign
DesignLanguage
CLASSIC_ICONS
ClassicDesign
data-design='classic'
data-design='studio'
data-design='paper'
t.design === 'dallaglio'
t.design !== 'dallaglio'
```

Historical documentation may retain these terms. Runtime source, active tests, and active verification scripts may not.

16.2 Automated evidence

- focused tests for changed providers and shared components;
- TypeScript without errors;
- ESLint on affected files, followed by the configured broader check;
- production Next.js build;
- route or application smoke checks;
- `git diff --check`.

16.3 Rendered evidence

At minimum, verify:

- signed-in desktop light and dark;
- signed-in phone-width light and dark;
- shell navigation, search, notifications, account menu, preferences, and dialogs;
- representative dense registers, cards, tables, filters, forms, charts, empty states, loading states, error states, and retries;
- keyboard focus, Escape behavior, touch-visible actions, and reduced motion;
- no Classic toggle, label, mark, glass, gradient, glow, icon weight, or constrained-width branch remains visible;
- no live business data is mutated solely for visual verification.

17. Source inventory

17.1 Primary Classic implementation

Area	Archived source
Design selection	components/shared/design-system/shared/design.ts
Theme and preference providers	components/shared/design-system/tokens.tsx
Classic icon meanings	components/shared/design-system/classic/icons.ts
Classic namespace marker	components/shared/design-system/classic/index.ts
Legacy Studio base	components/shared/design-system/bases/studio.css
Legacy Paper base	components/shared/design-system/bases/paper.css
Shared button branch	components/shared/design-system/Button.tsx
Shared close branch	components/shared/design-system/CloseButton.tsx
Shared primitive branches	components/shared/design-system/primitives.tsx
Shared component branches	components/shared/design-system/components.tsx
Icon selection	components/shared/design-system/DslIcon.tsx
Global icon preference	components/shared/design-system/icons.tsx
Chart branches	components/shared/design-system/charts.ts
Confirmation branch	components/shared/design-system/confirm.tsx
Pre-paint selection	app/layout.tsx
Global legacy styles	app/globals.css

17.2 App shell files with design branches

- components/app-shell/AppShell.tsx
- components/app-shell/AuthMenu.tsx
- components/app-shell/BottomBar.tsx
- components/app-shell/PreferencesPanel.tsx
- components/app-shell/SidebarNavigation.tsx
- components/app-shell/TopNavigation.tsx

17.3 Route and feature files with explicit design branches

At the archived snapshot, explicit design checks were found across 49 route or route-adjacent files:

- `app/admin/lists/page.tsx`
- `app/admin/page.tsx`
- `app/av/page.tsx`
- `app/availabilities/page.tsx`
- `app/availability/page.tsx`
- `app/breakdowns/analytics/page.tsx`
- `app/breakdowns/page.tsx`
- `app/competency/page.tsx`
- `app/compliance-register/page.tsx`
- `app/compressors/page.tsx`
- `app/condition-monitoring/page.tsx`
- `app/contractors/page.tsx`
- `app/documents/page.tsx`
- `app/drivers/page.tsx`
- `app/employees/page.tsx`
- `app/engineering_report/page.tsx`
- `app/engineering-dashboard/page.tsx`
- `app/equipment/page.tsx`
- `app/inventory/page.tsx`
- `app/issues/page.tsx`
- `app/job-cards/page.tsx`
- `app/leave-management/page.tsx`
- `app/leaves/page.tsx`
- `app/login/page.tsx`
- `app/near_miss/page.tsx`
- `app/noticeboard/page.tsx`
- `app/overtime/page.tsx`
- `app/pachedu/page.tsx`
- `app/ppe/allocate/page.tsx`
- `app/ppe/OrderListPanel.tsx`
- `app/ppe/page.tsx`
- `app/pto/page.tsx`
- `app/quotations/page.tsx`
- `app/reliability/page.tsx`
- `app/requisitions/page.tsx`
- `app/safety_complaints/page.tsx`
- `app/services/page.tsx`
- `app/sheq_inspection/page.tsx`
- `app/sheq/page.tsx`

- [app/shifts/page.tsx](#)
- [app/spares/import/page.tsx](#)
- [app/spares/page.tsx](#)
- [app/tasks-events/page.tsx](#)
- [app/timesheets/page.tsx](#)
- [app/timesheets/TimesheetDayCell.tsx](#)
- [app/timesheets/TimesheetEmployeeCell.tsx](#)
- [app/training/page.tsx](#)
- [app/vfl/page.tsx](#)
- [app/work_stoppage/page.tsx](#)

Additional shared and UI files brought the explicit branch count to 72. The exact count is a snapshot, not a completion metric; subsequent edits may move or remove branches.

17.4 Browser and test dependencies

Classic appeared in token tests, design persistence tests, icon-map coverage tests, visual tests, and route verification scripts. Several scripts switched to Classic for a smoke phase and then restored Dallaglio. Those phases should be removed only after Dallaglio-only runtime behavior is established.

18. Historical implementation record

The formal split was introduced in commit [df15837](#) with the intent that Classic and Dallaglio remain genuinely separate rather than using a superficial CSS overlay. That change added the design contract, dual icon registries, Dallaglio controls and shell, shared buttons and close controls, and many route branches.

Later commits expanded Dallaglio adoption and route audits. Full-width layouts were first applied broadly in [152f95f](#), then scoped to Dallaglio in [3b58f67](#) to avoid changing Classic. The archived source snapshot at [dd75b27](#) includes the later monochrome Dallaglio structural palette and documentation architecture.

This history explains why Classic logic is distributed across routes: compatibility was intentionally preserved during incremental migration. Retirement should now remove that compatibility deliberately and test each retained behavior.

19. Future-use guidance

Use this archive when:

- a future designer wants to understand why a shared component or interaction rule exists;
- a regression resembles one of the recorded failure modes;
- a new design language is considered and the team needs to separate behavior from appearance;
- an agent needs a checklist for proving that Classic has been completely removed;
- a removed visual concept inspires a new Dallaglio-native solution without reusing old runtime code.

Do not use this archive to:

- restore the Classic selector;
- copy Classic utility strings or glass effects into Dallaglio;
- justify duplicate route markup;
- treat historical screenshots or smoke tests as current acceptance evidence;
- preserve dead code “just in case.” Git history and this archive are the preservation mechanism.

20. Final preservation statement

Classic's greatest value was not glass, violet, gradients, or glow. Its lasting value was the move from scattered page styling toward shared contracts, consistent interaction, accessibility, semantic state, reusable components, and verifiable behavior.

The correct retirement therefore has two parts:

1. preserve and translate the principles documented here; and
2. remove every Classic runtime trace so Dallaglio becomes one coherent, maintainable system.

That approach protects the work without carrying its obsolete implementation forever.